

PRD: M-Wiki — LLM-Maintained Second Brain

Triage: ready-for-agent

Status: Greenfield build. Target repo is currently empty; this PRD specifies the system from scratch.

Stack: TypeScript + Mastra (agents, workflows, workspace: filesystem/bash/BM25). Discord interface. Obsidian-readable markdown vault synced via git to a private GitHub repo. Agent runs on a cloud VPS.

Problem Statement

I'm a PhD researcher with a steady stream of things worth keeping — papers, articles, PDFs, screenshots — across both my research and my personal life. Every time I find one, turning it into something useful means reading it, summarizing it, filing it somewhere sensible, linking it to what I already know, and recording where it came from. That bookkeeping is tedious, so most sources never get processed: they rot in browser tabs, downloads folders, and bookmarks. My knowledge stays scattered instead of compounding, and weeks later I can't find the thing I read or recall which source a claim came from.

I already keep two Obsidian vaults — one personal, one for the PhD — but they're human-maintained, so they suffer the same fate: the maintenance burden grows faster than the value, cross-references go stale, and the vaults drift. I want to hand a source to an assistant over Discord and have it reliably turned into a properly-filed, cross-linked, citable page in my second brain — without me doing the bookkeeping, without ever losing the original, and without ever being unable to trace where a claim came from.

Solution

A persistent, compounding second brain that an LLM agent builds and maintains for me. Instead of retrieving from raw documents at query time (RAG), the agent incrementally compiles my sources into a structured, interlinked wiki of markdown files and then keeps it current. When I add a source, the agent reads it, extracts what matters, and integrates it into the existing brain — creating or updating entity, concept, and map pages, recording provenance, flagging contradictions, and maintaining cross-references.

Concretely, from my perspective:

- I send a source to a **Discord** bot — paste a URL, upload a PDF, drop a screenshot — and it banks it into my raw collection with its provenance. (Obsidian Web Clipper is a secondary desktop path.)
- I tell the bot to **ingest** it. It reads the source, shows me the key takeaways and the pages it intends to touch, I confirm or redirect, and it writes the pages, updates the index and log, and commits — one tidy commit per source.
- I **ask questions** of the brain in Discord. It answers with citations that trace back to the exact source, and offers to file especially useful answers back as new pages so my explorations compound too.
- Periodically it **health-checks** the wiki on its own, posts a report, fixes the mechanical issues, and hands me a checklist of judgment calls.
- I read and browse everything in **Obsidian** (graph view, Dataview dashboards); the agent writes it. My laptop and phone stay in sync through git/GitHub.

The two vaults become one unified brain: personal and PhD live as tagged domains under one roof, so cross-domain connections — a psychology concept that shows up in both, a method paper that reshapes my workflow — can actually form.

User Stories

Capture

1. As a researcher, I want to paste a URL into Discord and have it saved as a clean markdown source with its provenance, so that I can bank an article in one step from anywhere.
2. As a researcher, I want to upload a PDF to Discord and have it stored in my raw collection with capture metadata, so that papers stop piling up unread in my downloads folder.
3. As a researcher, I want to drop a screenshot or image into Discord and have it stored and described/OCR'd, so that visual sources are captured and searchable.

4. As a researcher, I want a bare URL or file with no accompanying text to be auto-captured without ceremony, so that banking a source is frictionless.
5. As a researcher, I want captured web articles converted to readable markdown server-side, so that both the agent and I can read the source as text.
6. As a researcher, I want to keep using Obsidian Web Clipper on desktop and have those clippings ride git up to the agent, so that I'm not forced into a single capture path.
7. As a researcher, I want every captured source to record where it came from (original URL, capture date, source type, domain), so that I never lose the trail back to the original.
8. As a researcher, I want my original sources treated as immutable, so that the agent can never silently rewrite or destroy what I captured.

****Ingest****

1. As a researcher, I want to ask the agent to ingest a captured source, so that it becomes integrated knowledge rather than an inert file.
2. As a researcher, I want the agent to first show me the key takeaways and the list of pages it plans to create or update, so that I can confirm or redirect emphasis before it writes anything.
3. As a researcher, I want a single ingest to create a Source page and update the relevant Entity, Concept, and Map pages in one pass, so that a source touches everything it should without me chasing references.
4. As a researcher, I want the agent to record each claim with a precise locator back to the source (page number for PDFs, heading/anchor for articles) and a short verbatim quote for key claims, so that the wiki is citation-grade and any assertion is verifiable in seconds.
5. As a researcher, I want each ingest committed as one atomic git commit with a descriptive message, so that I can review or revert a source's entire footprint as a unit.
6. As a researcher, I want the ingest to update the index and append a log entry automatically, so that navigation and history stay current without my involvement.
7. As a researcher, I want to review the actual diffs later in Obsidian or on GitHub rather than in chat, so that chat handles decisions and git handles diffs.
8. As a researcher, I want each ingest to happen in its own Discord thread, so that a source's back-and-forth stays contained.

9. As a researcher, I want the agent to flag when a new source contradicts or supersedes an existing claim, so that the synthesis stays honest as it evolves.

****Query****

10. As a researcher, I want to ask natural-language questions of my brain in Discord, so that I can get answers synthesized from everything I've read.

11. As a researcher, I want every answer to carry citations that resolve to source pages and through them to the immutable originals, so that I can trust and trace any claim.

12. As a researcher, I want the agent to offer to file a substantive answer back as a Concept or Map page, so that my explorations compound instead of evaporating into chat history.

13. As a researcher, I want throwaway questions to stay ephemeral by default, so that the wiki doesn't fill with trivial Q&A.

14. As a researcher, I want richer answer formats on request — a comparison table, a Marp slide deck, a matplotlib chart, so that I can get the shape of answer the question calls for.

****Lint / maintenance****

15. As a wiki owner, I want a scheduled health-check that posts a report to Discord, so that the wiki stays healthy without me remembering to run it.

16. As a wiki owner, I want the health-check to auto-fix only mechanical issues (broken links, index/log drift, stubs for recurring ghost links) in one atomic commit, so that routine entropy is handled for me.

17. As a wiki owner, I want contradictions, likely-stale claims, merge candidates, and data gaps surfaced as a prioritized checklist for my approval, so that I stay in control of substantive judgment calls.

18. As a wiki owner, I want the agent to suggest new questions to investigate and sources to look for, so that the brain keeps getting richer.

****Navigation, sync, and reading****

19. As a researcher, I want a lean agent-maintained ``index.md`` in plain text, so that the agent can orient itself quickly and I get a birds-eye catalog with one-line summaries.

20. As a researcher, I want a chronological ``log.md`` with a consistent grep-able prefix, so that I (and the agent) can see how the brain evolved.

21. As a researcher, I want everything in a single Obsidian vault so wikilinks and graph view span the whole brain, so that I can see what's connected to what.

22. As a researcher, I want optional Dataview dashboards over page frontmatter, so that I can browse by type, domain, status, or date.

23. As a researcher, I want my laptop and phone to stay in sync through git/GitHub, so that I can read the brain anywhere and capture from my phone.

24. As a researcher, I want the agent's writes and my edits to touch disjoint paths (agent writes ``wiki/``, I write ``raw/``), so that sync stays effectively conflict-free.

****Domains and bootstrap****

25. As a researcher, I want personal and PhD content unified in one brain with a domain tag rather than split into separate vaults, so that cross-domain connections can form.

26. As a researcher, I want to start from a cold wiki seeded only with top-level Map pages for my main areas, so that the first ingests have structure to attach to without a big upfront migration.

27. As a researcher, I want my existing notes to remain available as immutable raw sources and be pulled into the wiki lazily when relevant, so that nothing is lost and I don't pay to process stale material.

****Schema evolution and cost****

28. As a wiki owner, I want the conventions to live in a single living ``AGENTS.md`` loaded at runtime, so that I can tune behavior by editing markdown without redeploying code.

29. As a wiki owner, I want the agent to propose changes to ``AGENTS.md`` as diffs I approve, so that the schema co-evolves without the agent silently rewriting its own rules.

30. As a wiki owner, I want cheap models used for mechanical steps and strong models reserved for judgment, so that frequent ingests and scheduled lints stay affordable without sacrificing quality where it matters.

****Developer-facing (system qualities)****

31. As a developer, I want the wiki domain isolated behind ports so Discord is just one driving adapter, so that I can test operations without Discord and add other transports later without touching the core.

32. As a developer, I want the agent to choose operations by selecting tools rather than matching keywords, so that intent routing is natural and handles compound requests.

33. As a developer, I want the mechanical steps of ingest/lint encoded as deterministic workflow steps, so that "always update the index / always commit" is a structural guarantee rather than a prompt's good intentions.

34. As a developer, I want every operation runnable end-to-end against a temp fixture vault with the LLM, Discord, and network stubbed, so that I can assert real filesystem and git outcomes deterministically.

Implementation Decisions

****Overall architecture — hexagonal (ports & adapters).**** The wiki domain (the four operations plus their Mastra workflows) is the core. It is reached only through a ****driving port**** and reaches the outside world only through ****driven ports****. Discord is one driving adapter; the LLM, git, filesystem, search, and network are behind driven ports.

- ****Driving port**** — a single transport-agnostic entry point that accepts a normalized ``InboundMessage`` and returns an ``AgentResponse``. Driving adapters: a ****Discord adapter**** (maps Discord events ↔ DTOs, manages threads), a ****test driver**** (feeds synthetic messages in tests), and a ****scheduler adapter**** (fires the lint operation on a cron cadence on the VPS).

- ****Driven ports**** — ``LLM`` (model calls), ``SourceFetcher`` (URL→markdown via Readability; PDF text extraction; image vision/OCR), ``Responder`` (send replies/threads), ``Vault`` (filesystem read/write), ``VCS`` (git add/commit/push),

`Search` (BM25). `Vault` / `VCS` / `Search` are backed by the Mastra workspace;
`LLM` / `SourceFetcher` / `Responder` are the boundaries stubbed in tests.

The DTOs at the driving port (shape encodes the decision; from the design discussion, trim to intent):

```ts

```
type InboundMessage = {
 text: string; // may be empty (e.g. a bare attachment)
 author: { id: string; name: string };
 context: { channelId: string; threadId?: string; ts: string };
 attachments: Attachment[]; // files + detected URLs, normalized
 replyTo?: string;
};
```

```
type Attachment =
 | { kind: 'url'; url: string }
 | { kind: 'file'; filename: string; mime: string; bytes: ArrayBuffer };
```

```
type AgentResponse = {
 text: string; // reply to send back
 citations: Citation[]; // source-page refs with locators
 files?: OutFile[]; // optional generated artifacts (Marp, chart)
 thread?: { create?: boolean; title?: string };
};
```

```

****Intent routing via tool selection (revises the earlier keyword-trigger decision).**** The core's conversational agent is given the operations as tools — `captureSource`, `ingestSource`, `queryWiki`, `runLint` — and routes by *choosing which tool(s) to*

call_ from the `InboundMessage`. There is no keyword matching. Compound requests are handled by composing tools.

- **Cost/safety guardrail:** `captureSource` (cheap, safe) and `queryWiki` (read-only) execute directly from the agent's decision. `ingestSource` and `runLint` mutate many files and therefore run the checkpoint first — the agent proposes a plan and waits for explicit user confirmation before any mutating write.

- **Optional optimization:** an `InboundMessage` whose `text` is empty and whose only content is a URL/file may short-circuit to `captureSource` without an LLM classification call.

Runtime decomposition (Mastra). Hybrid: **workflows** for ingest and lint (mutation-heavy, reliability-critical — mechanical steps are deterministic and unskippable: file writes, `index.md` update, `log.md` append, git commit/push; judgment steps are embedded agent calls); a **conversational agent** for capture and query. This makes the "touch 10–15 files consistently, always commit" promise structural, and gives observability/resumability on the heavy operations.

Hosting & sync. Agent runs on a **cloud VPS** holding the canonical vault. **Git** is the sync layer; the agent commits/pushes after each ingest and lint, clients pull (Obsidian Git plugin). The remote hub is a **private GitHub repo** (off-site backup, simple client setup). The ownership invariant — agent writes only `wiki/`, human writes only `raw/` — keeps the two sides on disjoint paths and merges effectively conflict-free.

Vault layout (single Obsidian vault, layer-first).

...

raw/ immutable sources — personal/, phd/, assets/ (human-owned)

wiki/ agent-owned pages — sources/, entities/, concepts/, maps/

AGENTS.md the living schema/conventions, loaded at runtime

index.md lean agent-maintained catalog

log.md append-only chronological record


```

Domain (personal vs phd) is a **frontmatter tag**, not a folder, so cross-domain pages need not pick a home.

**Page taxonomy** — four types, each a folder. ``Source`` (one per ingested item; summary, claims-with-locators, link to the ``raw/`` original), ``Entity`` (people, orgs, papers-as-objects, datasets, tools), ``Concept`` (ideas, methods, theories; the compounding permanent notes), ``Map/Overview`` (area maps; home of the evolving synthesis/thesis). Comparisons are Concept pages or filed query outputs; the thesis lives at the top of Map pages.

**Naming.** Title Case for Entity/Concept/Map (``Self-Supervised Learning.md``), with ``aliases`` for acronyms/variants. **Citekeys** for Source pages (``lecun2015.md``), full title in frontmatter — aligns with Zotero/BibTeX. The agent quotes paths in shell operations, so spaces are a non-issue.

**Frontmatter** — moderate, Dataview-ready, fixed small set:

```
```yaml
type: concept      # source | entity | concept | map
tags: [phd]        # domain(s): personal, phd, or both
created: 2026-06-09
updated: 2026-06-09
status: developing # stub | developing | stable
aliases: [SSL, self-supervised learning]
# Source pages additionally:
source_url: https://...
source_type: article # article | pdf | image
captured: 2026-06-09
```

```

No structured relationship fields — relationships live in body links.

**Linking.** Obsidian `[[wikilinks]]` on first meaningful mention; link liberally, including to not-yet-created pages (ghost nodes → lint backlog). The agent writes **forward links only**; backlinks are computed by Obsidian (zero reverse-link bookkeeping). Source pages link to their `raw/` original.

**Index & log.** `index.md` is plain text (the agent reads raw markdown, so a Dataview-generated index wouldn't be visible to it), grouped by type/domain with one-line summaries, updated every ingest; an optional separate Dataview dashboard page serves human browsing. `log.md` is append-only with a grep-able prefix (`## [2026-06-09] ingest | Title`), recording ingests, lints, and filed query outputs — not ephemeral Q&A.

**Capture pipeline.** Discord-first (bot saves into `raw/<domain>/` with provenance frontmatter; articles converted to markdown). Source types at launch: **text** (articles), PDF (text + figures via vision), images (vision + OCR). Audio/video deferred.

**Search.** Mastra workspace **BM25** plus filesystem/bash, complemented by `index.md` orientation, `aliases`, agent query-expansion, and wikilink traversal. Vector/hybrid is available in Mastra and can be enabled later if recall proves insufficient.

**Models — tiered by step.** Cheap/fast (Haiku-class) for mechanical steps (extraction, summarization, classification, formatting); strong (Sonnet-class) for judgment (page selection, drafting, query synthesis); optionally top-tier (Opus-class) for the heaviest lint (contradiction resolution). Per-step model selection via Mastra. Acceptable on-ramp: start Sonnet-class everywhere, split Haiku in once flows stabilize.

**Bootstrap.** Cold start with a one-session seed of top-level Map pages for main PhD/personal areas; existing notes remain immutable raw sources, ingested lazily on demand.

**Schema co-evolution.** Conventions live in a single `AGENTS.md` at the repo root, loaded into agent instructions and workflow prompts at runtime. The agent may propose edits to it as diffs the user approves; it never silently rewrites its governing rules.

## ## Testing Decisions

**What makes a good test here:** assert **external, observable behavior** — the files that exist after an operation, their frontmatter/links/provenance, the `index.md` / `log.md` deltas, the git commit, and the structured `AgentResponse` — never internal step order or which private function called which. The whole system has only **two external boundaries worth stubbing** (the `LLM` and the `transport/Responder/Discord`) plus outbound **network** (`SourceFetcher`). Everything else — `Vault` filesystem, `VCS` git, `Search` BM25 — runs **for real** against a temp fixture vault in each test, because those are exactly the effects we care about. This keeps tests at the highest seam while staying deterministic.

**Seams (highest first):**

- Driving-port seam (primary).** Feed an `InboundMessage` through the port and assert the resulting `AgentResponse` plus vault/git state, with `LLM` and `SourceFetcher` stubbed and `Vault` / `VCS` / `Search` real against a temp vault. Covers capture, ingest, query, and lint end-to-end as behavior. The ingest checkpoint is exercised by asserting that no mutating write occurs before confirmation.
- Discord adapter seam (thin).** Test only the mapping: Discord event → `InboundMessage`, and `AgentResponse` → Discord messages/thread actions. The decision logic lives in the core, so this stays small.
- Pure-transform seams.** Direct, fixture-based unit tests for the deterministic transforms: article→markdown, PDF/image extraction wrappers, citekey + frontmatter generation, provenance-locator extraction, `index.md` / `log.md` updaters, and the git-commit wrapper.
- LLM contract/eval seam.** For non-deterministic judgment steps, assert outputs against a **contract**, not exact prose: tool-selection routing picks the right operation for a given message; a drafted page has valid YAML frontmatter, resolvable wikilinks, citations with locators, and lands in the correct folder/type. Run over a small curated eval set; tolerate wording variation.

**Modules tested:** the four operation workflows/handlers via the driving port (seam 1); the Discord adapter mapping (seam 2); each pure transform (seam 3); the agent's router and page-draft contracts (seam 4). Driven-port fakes (``LLM``, ``SourceFetcher``, ``Responder``) are provided as in-memory test doubles; ``Vault`` / ``VCS`` / ``Search`` use a temp directory initialized as a git repo per test.

**Prior art:** none — greenfield. Establish the conventions with this feature: Vitest as the runner, a ``makeTempVault()`` fixture helper (temp dir + ``git init`` + seeded Map pages), and in-memory driven-port fakes. Mastra workflows are run directly in tests via their run API; agent steps are driven through the stubbed ``LLM`` port. These helpers become the prior art subsequent features copy.

## ## Out of Scope

- **Audio/video ingestion** (Whisper transcription, podcast/YouTube). Deferred; revisit when the need is real.
- **Full upfront backfill** of existing notes into the wiki. Bootstrap is cold-start + map seed; backfill is lazy/on-demand.
- **Vector/hybrid search.** BM25-primary at launch; hybrid is a later flip if recall slips.
- **Keyword-trigger command parsing.** Explicitly replaced by tool-selection intent routing.
- **Multi-agent router architecture.** The hybrid workflows-plus-agent design is the chosen runtime.
- **Non-Discord transports** (CLI, HTTP webhook, email capture). The ports make these cheap to add later, but only the Discord (and test + scheduler) adapters are built now.
- **Real-time sync** (Syncthing/Obsidian Sync). Git-on-a-cadence is the sync model.
- **Deployment/secrets hardening** beyond what's needed to run on the VPS (process manager, Discord token, model API keys, git credentials) — assumed, not specified here.
- **Migration tooling** to consolidate the two existing vaults into ``raw/`` — a one-time human setup step, not an agent feature.

## ## Further Notes

- **Provenance is the trust anchor.** Claim-level locators plus short key quotes are what make the brain citation-grade for actual paper-writing; keep extraction faithful and avoid over-quoting copyrighted material.
- **The checkpoint is also the safety net for routing.** Because mutating operations confirm before writing, an occasional misroute by the agent is caught before any file changes — this is what lets us drop keyword gating safely.
- **Obsidian-native affordances do real work for free:** computed backlinks (no reverse-link maintenance), graph view (shape of the brain), ghost links (lint backlog), Dataview (human dashboards from frontmatter). Lean on them rather than rebuilding them.
- **Code vs. content repos:** the Mastra application code should live separately from the vault repo (avoid `node\_modules` and build artifacts inside the Obsidian vault). The vault repo is the one synced to GitHub and pulled by clients.
- **Co-evolution loop:** expect `AGENTS.md` to change as conventions are discovered in use; the agent-proposes / human-approves diff flow is the mechanism, and the schema's git history is its changelog.
- **Future work seeds:** enable vector/hybrid search; add audio/video ingestion; add a CLI or HTTP driving adapter; build Dataview dashboards; wire Marp/chart generation for query outputs.